OmniSight: Multimodal UI Self-Healing & RPA Agent

Project Definition & Execution Plan | Prepared by Juhi Thacker | 5-week full-scope build

1. Project Summary

OmniSight is an RPA (Robotic Process Automation) agent that keeps working when the user interface changes. A normal automation bot breaks the moment a button is renamed, moved, or covered by a pop-up. OmniSight combines DOM analysis with a multimodal vision model so it can find the right element by what it *is* and *looks like*, verify every step, and repair its own broken steps automatically. Every repair is logged, scored, and shown on a dashboard.

2. Problem Statement

UI automation scripts depend on fixed selectors such as IDs, CSS classes, and XPaths. When a website or app is updated, those selectors change and the automation fails. Teams then spend time fixing scripts by hand, which is costly and repetitive.

OmniSight's goal: build an automation agent that detects a failure, understands the screen the way a person would, finds the intended element again, completes the task, and remembers the fix for next time.

3. Objectives

1. Run multi-step workflows defined in a simple JSON format.
2. Locate UI elements by natural-language description, with no hard-coded selector required.
3. Verify after every step that the expected state was reached.
4. Heal broken steps using a ladder of strategies, from cheap DOM methods to vision.
5. Remember successful fixes (locator memory) so the same failure is cheap the next time.
6. Accept a plain-English goal and turn it into an executable plan.
7. Provide a dashboard with run history, screenshot timeline, and a healing report.
8. Measure the benefit with a benchmark: plain bot vs DOM healing vs full OmniSight.

4. Full Scope

Nothing is cut. The scope is ordered so the core is built first and extras last.

Core (Weeks 1-3)

- RPA engine: JSON workflow format and step runner (click, type, select, wait, extract)
- Multimodal perception: DOM simplifier, vision element finder, step verifier
- Self-healing ladder, blocker handling, locator memory, confidence scoring, human escalation
- Natural-language planner
- Safety guardrails
- Dashboard and metrics

Extras (Weeks 4-5)

- Record-and-replay with multi-locator capture
- Chrome extension (Manifest V3)
- Desktop support (thin proof of concept)
- Benchmark, documentation, demo video, and project report

5. Fixed Technology Decisions

Layer	Choice
Browser automation	Playwright (Node.js with TypeScript planned, matching existing Node/Express experience)
Desktop automation	robotjs (Node) or PyAutoGUI (Python), using the same screenshot-and-vision loop
Vision and planning	Hosted multimodal model API, wrapped in one function so the provider can be swapped
Backend	Express (or FastAPI)
Storage	SQLite for logs, runs, and locator memory
Dashboard	React + Tailwind with a component library
Chrome extension	Manifest V3, reusing the backend
Test application	Own app (Shoes Mart planned) so UI mutations are fully controllable

Hardware note: the development laptop has 8 GB RAM, so no local vision models are run. All vision and planning calls go to a hosted API.

6. System Architecture

OmniSight is built as a pipeline of cooperating modules:

1. **Planner** converts a natural-language goal into a list of workflow steps.
2. **Step Runner** executes each step (click, type, select, wait, extract) using Playwright.
3. **Perception layer** builds a simplified DOM (visible interactive elements with text, role, and position) and, when needed, sends a screenshot plus element descriptions to the vision model.
4. **Verifier** checks after each step whether the expected state was reached.
5. **Healer** runs when a step fails and climbs the healing ladder (see section 7).
6. **Locator Memory** stores old and new locators after a successful heal and tries the healed one first next time.
7. **Safety layer** enforces step limits, loop detection, confirmation for destructive actions, and masking of sensitive data.
8. **Backend API and SQLite** store runs, steps, screenshots, heals, and metrics.
9. **Dashboard** displays run history, screenshot timeline, healing reports, metrics, and the human review queue.
10. **Extensions** (record-and-replay, Chrome extension, desktop mode) feed workflows and runs into the same engine and backend.

Key principle: heal only on failure. Calling the vision model on every step wastes time and money.

7. Self-Healing Ladder

Strategies are tried in order, cheapest first:

1. Retry with a wait
2. Fallback locators: text, aria-label, role
3. Fuzzy DOM match
4. Vision-based re-location using a screenshot
5. Blocker handling: pop-ups, modals, cookie banners, overlays
6. Human escalation with a screenshot when everything fails

Every successful heal records the strategy used, the old and new locator, and a **confidence score**. Low-confidence heals go to a human review queue.

8. Safety Requirements

- Maximum step limit per run
- Loop detection
- Confirmation before destructive actions (delete, pay, submit)
- Passwords and sensitive fields masked in screenshots and logs

9. Test Application and Mutation List

A test app that I fully control is essential. Planned: **Shoes Mart**. Planned demo workflow: open the site, log in, search a product, add it to the cart.

Mutations to apply to the test app (each one is also a benchmark case):

1. Renamed element ID
2. Changed CSS class
3. Changed button text
4. Moved button to a different position
5. Changed element order in the layout
6. Changed colors and styling
7. Wrapped element in a new container
8. Pop-up or modal appearing on load
9. Cookie banner covering the page
10. Overlay blocking a button

The benchmark will use 10-15 mutations and compare three setups: **plain bot**, **DOM healing only**, and **full OmniSight with vision**, recording success rate, time per run, and cost per run.

10. Five-Week Roadmap

Week 1: Engine + Perception

Days 1-2: Setup and RPA engine

- Repo, folder structure, problem statement, demo workflow
- JSON workflow format and step runner
- Screenshot after every step, SQLite logging

Days 3-4: Run and break the workflow

- Run the demo workflow end to end with plain selectors
- Create the first UI mutations to prove the bot breaks

Days 5-7: Perception layer

- DOM simplifier
- Screenshot plus element description to the model, returning coordinates or the matching element
- Verifier after each step

Done when: the bot finds “the Login button” with no selector and verifies each step.

Week 2: Self-Healing Engine

Days 1-2: DOM-based healing ladder

- Retry with wait, fallback locators, fuzzy DOM match

Days 3-4: Vision recovery and blockers

- Vision-based re-location when DOM methods fail

- Pop-up, modal, cookie-banner, and overlay detection and dismissal

Day 5: Locator memory

- Store old and new locator after a successful heal, try the healed one first next time
- Confidence score for every heal

Days 6-7: Escalation and stress test

- Human-escalation path with screenshot
- Test against 10+ mutations

Done when: UI changes don’t stop the bot, and every heal is logged with strategy and confidence.

Week 3: Planner + Safety + Dashboard

Days 1-2: Planner

- Natural-language goal becomes a step list, executed with the verifier and healer active

Day 3: Safety

- Step limit, loop detection, confirmation for destructive actions, masking of sensitive fields

Days 4-7: Dashboard

- Backend API for runs, steps, heals, and metrics
- React dashboard: run history, screenshot timeline, healing report
- Metrics: success rate, heal rate, average heal time, cost per run
- Human review queue for low-confidence heals

Done when: a typed goal runs and produces a full healing report.

Week 4: Extras (Record-and-Replay + Chrome Extension)

Days 1-3: Record-and-replay

- Record browser actions and auto-generate the JSON workflow
- Capture several locators per element (ID, text, role, screenshot crop)

Days 4-7: Chrome extension

- Record and run workflows from the browser
- Send runs to the backend so they appear in the dashboard
- Show live status of the current step

Done when: a workflow can be recorded once, replayed, and viewed in the dashboard.

Week 5: Desktop Support + Benchmark + Submission

Days 1-2: Desktop support (thin version)

- Screenshot, vision model, click and type through robotjs or PyAutoGUI
- Prove it with one desktop app workflow, for example Notepad
- Deliberately a thin proof, because it is the riskiest item

Days 3-4: Benchmark

- 10-15 mutations across the three setups
- Record success rate, time per run, and cost per run

Days 5-6: Documentation and demo

- README with architecture diagram

- 3-5 minute demo video
- Project report: problem, architecture, results, limitations

Day 7: Rehearsal

- Demo run-through and viva or interview Q&A preparation

Done when: the benchmark results, demo video, and report are complete.

Weekly Summary

Week	Focus	Checkpoint
1	Engine + perception	Bot finds elements by description
2	Self-healing	Bot survives UI changes
3	Planner + dashboard	Goal in, healing report out
4	Record-replay + extension	Record once, replay, view in dashboard
5	Desktop + benchmark + docs	Results, demo, report

11. Proposed Repository Structure

```
omnisight/
  engine/       # step runner, workflow JSON
  perception/   # DOM simplifier, vision calls
  healing/      # ladder, locator memory
  planner/
  server/       # Express API + SQLite
  dashboard/    # React
  test-app/     # Shoes Mart or link to it
  docs/         # problem statement, devlog
```

Keep `docs/devlog.md` with one short entry every day: what broke and how it was fixed. It becomes the project report.

12. Working Rules

- **Core first, extras last.** Weeks 1-3 are the project. Weeks 4-5 add to it.
- **Heal only on failure.**
- **Don’t build what you can install.** Use Playwright’s built-in locators and codegen, and a component library for the dashboard.
- **Time-box everything.** If a task takes double its slot, move on and return during a buffer.
- **Commit daily** and keep the devlog.
- **Test with mutations from Week 1,** not only in Week 5.
- Use an AI coding assistant for boilerplate (UI, CRUD, tests) so personal time goes to the healing logic.

13. Risks and Honest Limitations

- The plan is about 30 working days of effort compressed into 35 calendar days. Skipping days will break it, and it needs roughly 5-6 focused hours daily including weekends.
- Desktop support and the Chrome extension are the likeliest to overrun. If behind at the start of Week 4, build simplified versions of those two so the full scope still appears in the report.
- Vision model calls cost money and add latency, which is why healing runs only on failure.
- The benchmark will cover everything finished, and the report must state clearly which parts are proofs of concept.

14. First Actions (Days 1-2)

1. Create the repo and folder structure, then make the first commit.
2. Initialize the Node.js/TypeScript project and install Playwright.
3. Write the demo workflow as JSON.
4. Build the step runner with a screenshot after each step and SQLite logging.
5. Run the workflow end to end on the test app.
6. Add the first mutation and confirm the bot breaks.